GOA
ELECTRONICS
LIMITED

WEBFORM DESIGN STANDARDS

GOA ELECTRONICS LIMITED
A Subsidiary of EDC Ltd. (A Government of Goa Undertaking)

Important: The information contained in this document is for exclusive use of Goa Electronics Limited. The information contained in this document should not be passed on to any third party in part or full.

REVISION HISTORY

Revision No. / Version No.	Date	Prepared by	Approved by	Details of Change
1.0	07-Aug-2024	Austin De Sa (SSD)	Anant Yende (CTO)	

TABLE OF CONTENTS

1. Introduction

Designing effective webforms is crucial for user experience and can significantly impact the efficiency of data collection. By adhering to these standards, developers can create webforms that are user-friendly, efficient, and accessible, ultimately improving the overall user experience and data collection process.

2. Design

2.1 User Experience (UX)

- Keep the form as short and simple as possible. Only ask for essential information.
- Arrange fields in a logical sequence that aligns with user expectations and the flow of the task for example state, district, taluka, and village should be in one row.
- Provide real-time feedback to users as they complete each field instead of giving feedback after the user moves to another field.
- Provide help text or placeholder text within the input fields for additional guidance.
- For lengthy forms, break them into manageable tabs or sections.
- Mention information about allowed characters in the fields wherever applicable and should be as per the Validation Matrix.
    1. Name should contain letters, dot, apostrophe, @ and space.
    2. Remarks should contain only letters, numbers, dot, apostrophe, @, &, comma, hyphen, slash, space, and parentheses.

2.2 Accessibility

- Ensure all form fields have associated labels.
- Provide clear and specific error messages that are easy to understand and correct.
- Provide tooltip and alternate text for icons like edit, delete, and add buttons or links.
- Ensure the form's tab order follows the logical reading order.
- After form submission, provide clear confirmation messages and next steps.
- Before deleting provide confirmation alert.

2.3 Visual Design

- Use consistent styles for form elements to maintain visual hierarchy.
- Ensure adequate spacing between fields and consistent alignment to enhance readability.
- Use colors consistently to indicate states (e.g., red for errors).
- The visual presentation of text and images of text has a contrast ratio of at least 4.5:1, except for the following:

a. Large Text: (18 pt. or 14 pt. bold) Large-scale text and images of large-scale text have a contrast ratio of at least 3:1.

b. Incidental: Text or images of text that are part of an inactive user interface component, that are pure decoration, that are not visible to anyone, or that are part of a picture that contains significant other visual content, have no contrast requirement.

c. Logotypes: Text that is part of a logo or brand name has no contrast requirement.

- Ensure input field borders have sufficient contrast.
- Display icon within fields to help user understand the field type. Example. Date calendar icon, Rupee icon etc.

2.4 Form Fields

- Use appropriate HTML5 input types (Email, Tel, Date, etc.) for better user experience and validation.
- Adjust the length of input fields to match expected input length (e.g., shorter fields for zip codes, longer for comments).
- Dropdowns vs. Radio Buttons: Use dropdowns for long lists and radio buttons for a few options like Yes/No.
- Sensitive content should be masked before showing.
- Provide a clear and concise title at the top of the modal to inform users of the form's purpose.
- Ensure the title styling matches the overall theme of application for consistency.
- Design the modal to be appropriately sized for its content, ensuring it is neither too cramped nor excessively large. For example, modal can have 70% width. Consider modal fields before planning the %.
- Use a dimmed overlay behind the modal to focus user attention on the form and reduce distractions.
- If the form submission involves asynchronous operations (e.g., API calls), provide a loading indicator within the modal to inform users that their action is being processed.
- If the form is lengthy, enable scrolling within the modal while keeping the title and action buttons fixed for easy access.
- Prevent the background content from scrolling when the modal is open to maintain focus on the modal content.
- Display skeleton versions of input fields and buttons where data from API calls will be inserted.
- Ensure that skeleton screens do not block interaction with other parts of the form or application, maintaining overall usability.
- Reset fields which are applicable only in certain conditions.
- Prevent duplicate / repetitive hits / events.

- Always show loader when some background operation is on.
- When designing a webform, use a grid layout where the main form fields (such as name, mobile, and other inputs) should have a column span of 8 (for Ant Design) and 4 (for Bootstrap). Fields accepting more characters (such as Address, remarks, email, and others) must be set to a column span of 16 or 24 (for Ant Design) and 8 or 12 (for Bootstrap). This allocation ensures that the main fields have adequate space for user input while maintaining a balanced layout.

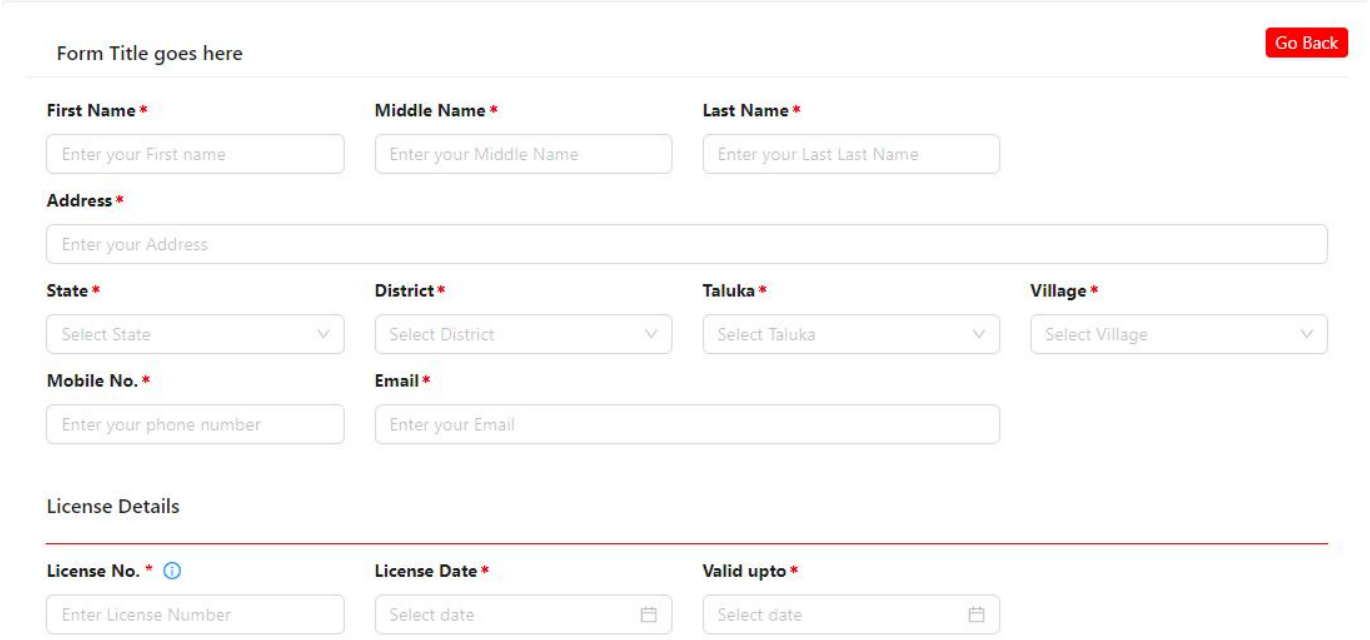

Fig 1. Sample Layout

2.5 Grid Design

- Adjust the grid spans for different screen sizes using media queries to ensure that the form remains user-friendly and readable on all devices. For example, the fields might span all 24 columns on mobile devices to provide sufficient input space.
- Utilize grid properties to center elements or distribute them evenly across the form layout.
- Use grid spans to align elements consistently within the form, maintaining a structured and organized appearance.
- Allow users to select multiple records using checkboxes for bulk actions.
- Show or hide action buttons based on user permissions to prevent unauthorized actions.
- Allow users to perform multiple actions in row grid with button shown as icons and tooltips defining the functionality.

- For more complex edits, use a modal or form that appears when the edit button is clicked, offering a detailed view for modifications.

2.6 Validation and Error Handling

- Clearly indicate required fields using asterisks or other markers.
- Provide specific and actionable feedback for validation errors.
- Ensure that users do not lose their data if there are validation errors by preserving input values.
- Load non-essential elements only when needed.

2.7 Mobile-Friendly Design

- Design forms that are responsive and work well on various screen sizes.
- Ensure form controls are large enough to be easily tapped on mobile devices.

3. Security

3.1 Disable Right Click

Disabling right-click on a web page is a common practice to prevent users from accessing context menus, copying content, or performing other actions. Similarly, developer option shortcuts such as Ctrl + Shift + I, F12 key too should be disabled.

3.2 Disable Back Button & Browser History

- Back button must be provided on the form to go from current page to previous page.
- Browser back button must be disabled and so the browser history to prevent unauthorised access to the pages.
- The page should not show any data from the cache if there is no network. (Example. Showing info from browser history)

3.3 Page Refresh

Necessary actions or conditions needs to be handled when the page is refreshed in the browser. Either the same page loads with the same data or the user is alerted about loss of information confirmation if proceeds with action (refresh).

3.4 Session Timeout

- Handling session timeouts is important for maintaining security and providing a good user experience.
- Session timeout is a period of inactivity after which a user's session is automatically ended, requiring them to re-authenticate.
- Standard session timeout to be set to 20 minutes.

3.5 Sessions

- A session is a mechanism for storing data that persists across multiple requests from the same user.
- This data once stored can be used until the session is terminated. Once the user logs out of the application, the session data must be terminated.
- Use Secure Cookies: Set HttpOnly, Secure, and SameSite attributes on cookies to protect session data.
- Avoid storing sensitive data directly in sessions. Instead, use session IDs to reference sensitive information stored securely on the server.
- Validate sessions on each request to ensure they are still valid and active.
- Change session IDs frequently to avoid misuse.

3.6 Transition between pages

- Transition between pages using relative URL.
- Avoid passing sensitive data using Query String.
- Always encrypt Query String values.
- Encrypting the data while transitioning from one page to another is a must.
- If data is passed in the URL, the values must be encrypted and the query string should be randomized value and encoded.
- Use Query String names which are not user understandable.
- Always encode URLs using HTML encode.
- Use POST method whenever possible.
- Do not send huge data in Query String.
- Pass limited information between pages.
- If passing data to some different software application, use token-based string using APIs.
- Be cautious about refresh or reload and URL stealing.

3.7 Content Security Policy (CSP)

Protects against Cross-Site Scripting (XSS) and data injection attacks by specifying which sources of content are allowed to be loaded.

This value must be set to "default-src https: data: 'unsafe-inline' 'unsafe-eval'.

3.8 Strict-Transport-Security (HSTS)

Forces browsers to interact with the server only over HTTPS, preventing SSL stripping attacks. The max-age must be set to 31536000 and includeSubDomains & preload property has to be set.

- max-age: Time in seconds for which the browser should enforce HTTPS.
- includeSubDomains: Apply HSTS policy to all subdomains.

- preload: Indicates that your domain should be included in the HSTS preload list maintained by browsers.

3.9 X-Frame-Options

Prevents clickjacking attacks by controlling whether a page can be framed by other sites. X-Frame-Options value must be set to SAMEORIGIN.

- DENY: Prevents the page from being framed.
- SAMEORIGIN: Allows framing only by pages from the same origin.

Ensure framing is disabled on the application.

3.10 X-Content-Type-Options

Prevents MIME type sniffing by enforcing that the browser should respect the Content-Type specified by the server.

X-Content-Type-Options value must be set to nosniff.

3.11 X-XSS-Protection

Provides basic protection against XSS attacks by enabling the browser's built-in XSS filter. X-XSS-Protection must be set by enabling it by setting the value to 1 and mode set to block.

- 1: Enables the XSS filter.
- mode=block: Prevents the browser from rendering the page if an XSS attack is detected.

3.12 Referrer-Policy

Controls the amount of referrer information sent with requests.

- no-referrer: No referrer information is sent.
- origin: Only the origin is sent as the referrer.
- strict-origin-when-cross-origin: Sends full referrer to same-origin requests, and only origin to cross-origin requests.

Ensure applications uses whitelist origins only.

3.13 Feature-Policy (now Permissions-Policy)

Controls which feature and APIs can be used in your web application.no-referrer: No referrer information is sent.

- geolocation=(self): Only allows geolocation access from the same origin.
- microphone= (): Disables microphone access.

3.14 Cache-Control

Directs how caching is handled by browsers and intermediary caches, preventing sensitive data from being cached.

- no-store: Prevents caching of sensitive information.
- no-cache: Forces revalidation with the server before using cached content.

Ensure no-store and no-cache is added on the pages of the application / for the whole application.

3.15 Access-Control-Allow-Origin (CORS)

Controls which origins are permitted to access resources on your server, protecting against Cross-Origin Resource Sharing (CORS) issues. Depending on the requirement specific URLs can be allowed, otherwise the default URL can be set to the applications URL.

- *: Allows all origins (use with caution).
- https://example.com: Allows only the specified origin.

Define origin list clearly and use on base or master page to avoid CORS errors.

3.16 Set-Cookie

Securely configure cookies to enhance security.

- HttpOnly: Prevents JavaScript access to the cookie.
- Secure: Ensures the cookie is only sent over HTTPS.
- SameSite: Controls whether the cookie is sent with cross-site requests.

HttpOnly and Secure value must be set to true and SameSite value must be set to strict.

3.17 Cross-Site Request Forgery (CSRF)

Unique tokens are included in requests to verify that the request originated from the authenticated user's session.

3.18 Concurrent Login

Prevent concurrent access to the site for the same user on same site. When the second user logs into the system using same credentials, the first user must get logged out and a message should be shown informing that, a login has happened elsewhere and if it is not you, consider changing your password. This is achieved by maintaining login session in the database and check if valid on every request.

3.19 SQL Injection

- Use Prepared Statements (Parameterized Queries): This ensures that user input is treated as data and not executable code.
- Stored Procedures: Encapsulate SQL logic inside stored procedures, which can also be parameterized.
- Escape User Input: Properly escape any user input before including it in SQL queries. However, this is less secure than parameterized queries.
- Regular Security Testing: Regularly test your application for vulnerabilities using automated tools and manual penetration testing.

4. Performance

Before going live, the web application must be complied to Security and Performance issues. Server hardening must be done and website must be tested on SSL labs & Security headers.

4.1 Key Performance Metrics

- GTMetirx Metrics: Ensure the page load time is under 3 seconds.
- Google PageSpeed Insights Metrics: Achieve a performance score of 90 and above for both mobile and web browser.

4.2 Optimization Strategies

- Optimize images using compression tools and use latest file formats.
- Minify JavaScript and CSS files to save on network and server bandwidth.
- Implement lazy loading for images and videos.
- Enable browser caching and server-side caching for content which does not change. Example. CSS, JS, images etc.
- Optimize database queries and reduce server load.
- Ensure mobile responsiveness and fast loading times on all devices.
- Update libraries and framework to their latest stable version.
- Check for updates to all libraries and frameworks used in the website.

4.3 Google Analytics & Google Search

- Configure the site with Google Analytics account cms.with.gel@gmail.com.
- Google Analytics helps gain deep insights into your website or app's performance, user behavior, and the effectiveness of your marketing efforts.
- Configure site as well as Google Search console to ensure site is listed on top and restricted pages are not listed on Google.
- For standalone web application ensure to do basic SEO to improve listing on Google. For more details on SEO refer Website Search Engine Optimization Standards.